

■ 2.3.7 Advanced Topic: Functions with Optional Arguments

When you define a complicated function, you will often want to have some of the arguments of the function be “optional”. If you do not give those arguments explicitly, you want them to take on certain “default” values.

Built-in *Mathematica* functions use two basic methods for dealing with optional arguments. You can choose between the same two methods when you define your own functions in *Mathematica*.

The first method is to have the meaning of each argument determined by its position, and then to allow one to drop arguments, replacing them by default values. Almost all built-in *Mathematica* functions that use this method drop arguments from the end. For example, the built-in function `Flatten[list, n]` allows you to drop the second argument, which is taken to have a default value of `Infinity`.

You can implement these kind of “positional” arguments using `_:` patterns.

`f[x_, k_:kdef] := value` a typical definition for a function whose second argument is optional

Defining a function with positional arguments.

This defines a function with an optional second argument. When the second argument is omitted, it is taken to have the default value `Infinity`.

```
In[1]:= f[list_, n_:Infinity] := f0[list, n]
```

Here is a function with two optional arguments.

```
In[2]:= fx[list_, n1_:1, n2_:2] := fx0[list, n1, n2]
```

Mathematica assumes that arguments are dropped from the end. As a result `m` here gives the value of `n1`, while `n2` has its default value of 2.

```
In[3]:= fx[k, m]
Out[3]= fx0[k, m, 2]
```

The second method that built-in *Mathematica* functions use for dealing with optional arguments is to give explicit names to the optional arguments, and then to allow their values to be given using transformation rules. This method is particularly convenient for functions like `Plot` which have a very large number of optional parameters, only a few of which usually need to be set in any particular instance.

The typical arrangement is that values for “named” optional arguments can be specified by including the appropriate transformation rules at the end of the arguments to a particular function. Thus, for example, the rule `PlotJoined->True`, which specifies the setting for the named optional argument `PlotJoined`, could

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

appear as `ListPlot[list, PlotJoined->True]`.

When you set up named optional arguments for a function f , it is conventional to store the default values of these arguments as a list of transformation rules assigned to `Options[f]`.

<code>f[x_, opts___] := value</code>	a typical definition for a function with named optional arguments
<code>name /. {opts} /. Options[f]</code>	replacements used to get the value of a named optional argument in the body of the function

Named arguments.

This sets up default values for two named optional arguments `opt1` and `opt2` in the function `fn`.

```
In[4]:= Options[fn] := { opt1 -> 1, opt2 -> 2 }
```

This gives the default value for `opt1`.

```
In[5]:= opt1 /. Options[fn]
Out[5]= 1
```

The rule `opt1->3` is applied first, so the default rule for `opt1` in `Options[fn]` is not used.

```
In[6]:= opt1 /. opt1->3 /. Options[fn]
Out[6]= 3
```

Here is the definition for a function `fn` which allows zero or more named optional arguments to be specified.

```
In[7]:= fn[x_, opts___] := k[x, opt2/.{opts}/.Options[fn]]
```

With no optional arguments specified, the default rule for `opt2` is used.

```
In[8]:= fn[4]
Out[8]= k[4, 2]
```

If you explicitly give a rule for `opt2`, it will be used before the default rules stored in `Options[fn]` are tried.

```
In[9]:= fn[4, opt2->7]
Out[9]= k[4, 7]
```